# MQTT 通信协议案例

## Revision History

| Draft Date | Revision No. | Description |
|---|---|---|
| 2024/06/25 | V1.0 | 1. 初始版本。 |

# 目　　录


前　言 ..... 3

1　MQTT 通信协议简介 ..... 4

1.1　概述 ..... 4

1.2　应用场景 ..... 5

1.3　Mosquitto 工具安装 ..... 5

2　mqtt_client 案例 ..... 6

2.1　案例说明 ..... 6

2.2　案例测试 ..... 6

2.2.1　评估板发布/上位机订阅 ..... 7

2.2.2　评估板订阅/上位机发布 ..... 8

2.3　案例编译 ..... 9

2.4　关键代码 ..... 10

3　mqtt_sinewave_pub 案例 ..... 11

3.1　案例说明 ..... 11

3.2　案例测试 ..... 12

3.3　案例编译 ..... 14

3.4　关键代码 ..... 15

更多帮助 ..... 17

# 前 言

本指导文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu 20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71

Linux SDK：Real-Time Edge Software 2.5

本文主要介绍基于 MQTT 通信协议开发案例，主要包括 mqtt_client 案例和 mqtt_sine wave_pub 案例，位于产品资料“4-软件资料\Demo\mqtt-demos\”目录下。

# 1 MQTT 通信协议简介

## 1.1 概述

MQTT（Message Queuing Telemetry Transport，消息队列遥测传输协议），是一种基于发布/订阅(Publish/Subscribe)模式的“轻量级”通讯协议，该协议构建于 TCP/IP 协议上，由 IBM 在 1999 年发布。MQTT 最大优点在于，可以极少的代码和有限的带宽，为连接远程设备提供实时可靠的消息服务。

MQTT 是轻量、简单、开放和易于实现的，同时作为一种低开销、低带宽占用的即时通讯协议，使其在物联网、小型设备、移动应用等方面有较广泛的应用。

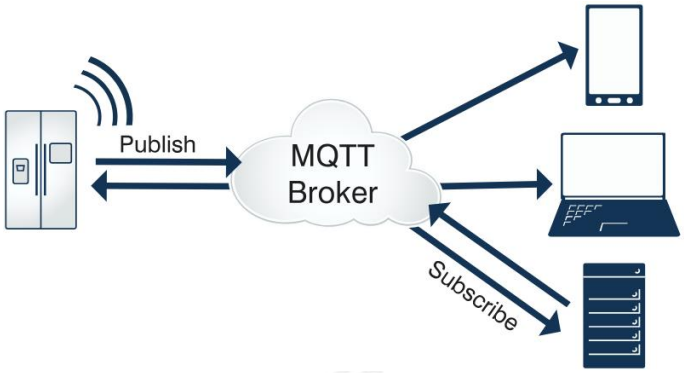


图 1

MQTT 具有如下特点：

（1） 轻量可靠：MQTT 的报文格式精简、紧凑，可在严重受限的硬件设备和低带宽、高延迟的网络上实现稳定传输。

（2） 发布/订阅模式(Publish/Subscribe)：发布/订阅模式的优点在于发布者与订阅者的解耦，实现异步协议。即订阅者与发布者无需建立直接连接，亦无需同时在线。

（3） 为物联网而生：提供心跳机制、遗嘱消息、QoS 质量等级 + 离线消息、主题和安全管理等全面的物联网应用特性。

（4） 生态更完善：覆盖范围广，已成为众多云厂商物联网平台的标准通信协议。

### 1.2 应用场景

MQTT 作为一种低开销，低带宽占用的即时通讯协议，可以极少的代码和带宽为联网设备提供实时可靠的消息服务，适用于硬件资源有限的设备及带宽有限的网络环境。常见的应用场景如下：

（1） 物联网 M2M 通信，物联网大数据采集。

（2） 移动即时消息及消息推送。

（3） 智能硬件、智能家居、智能电器。

（4） 车联网通信，电动车站桩采集。

（5） 智慧城市、远程医疗、远程教育。

（6） 电力能源、石油能源。

### 1.3 Mosquitto 工具安装

Mosquitto 是一款开源的 MQTT 消息代理（服务器）软件，提供轻量级的、支持可发布/可订阅的的消息推送模式。我司提供的评估板文件系统已支持 Mosquitto 工具，本文 mqtt_client 案例采用 Mosquitto 工具演示 MQTT 通信协议的通信功能。由于上位机 Ubuntu 系统作为通信对象，因此需在 Ubuntu 终端执行如下命令安装 Mosquitto 工具。

**Host#** sudo apt-get install mosquitto-clients

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install mosquitto-clients
[sudo] password for tronlong:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libmosquitto1
The following NEW packages will be installed:
  libmosquitto1 mosquitto-clients
0 upgraded, 2 newly installed, 0 to remove and 132 not upgraded.
Need to get 105 kB of archives.
After this operation, 442 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://cn.archive.ubuntu.com/ubuntu focal/universe amd64 libmosquitto1 amd64 1.6.9-1 [45.9 kB]
Get:2 http://cn.archive.ubuntu.com/ubuntu focal/universe amd64 mosquitto-clients amd64 1.6.9-1 [58.8 kB]
Fetched 105 kB in 2s (66.7 kB/s)
Selecting previously unselected package libmosquitto1:amd64.
(Reading database ... 192484 files and directories currently installed.)
Preparing to unpack .../libmosquitto1_1.6.9-1_amd64.deb ...
Unpacking libmosquitto1:amd64 (1.6.9-1) ...
Selecting previously unselected package mosquitto-clients.
Preparing to unpack .../mosquitto-clients_1.6.9-1_amd64.deb ...
Unpacking mosquitto-clients (1.6.9-1) ...
Setting up libmosquitto1:amd64 (1.6.9-1) ...
Setting up mosquitto-clients (1.6.9-1) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9.9) ...
tronlong@tronlong-virtual-machine:~$
```

图 2

# 2 mqtt_client 案例

## 2.1 案例说明

案例功能：使用 libmosquitto(MQTT version 3.1.1 client library)的 API 与 MQTT 代理服务器通信。基于 MQTT 通信协议，实现发布和订阅消息功能。

程序流程图如下图所示。

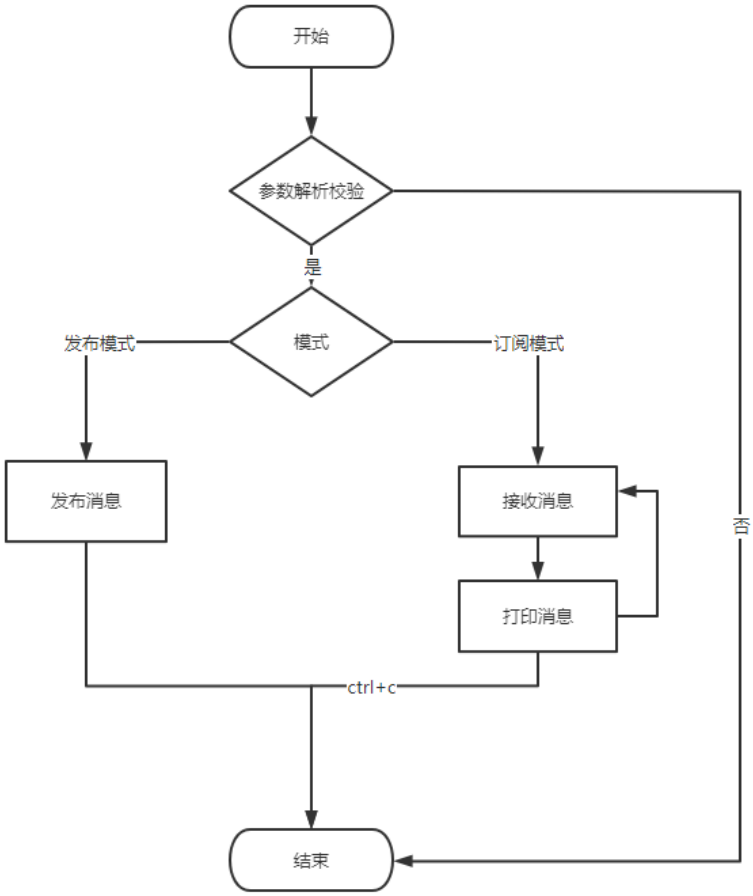


图 3

## 2.2 案例测试

本案例使用公网 MQTT HiveMQ 服务器与上位机 Ubuntu Mosquitto 工具通信。请通过网线将评估板千兆网口 ETH1 和上位机连接至公网，确保可正常访问互联网。

下表提供了可用的在线公共 MQTT 服务器，可根据需要自行切换。

表 1

| 服务器名称 | Broker 地址 | TCP 端口 | WebSocket |
|---|---|---|---|
| HiveMQ | broker.hivemq.com | 1883 | 8000 |
| Mosquitto | test.mosquitto.org | 1883 | 80 |
| Eclipse | mqtt.eclipseprojects.io | 1883 | 80/443 |
| EMQ X（国内） | broker-cn.emqx.io | 1883 | 8083/8084 |

评估板启动，将案例 bin 目录下 mqtt_client 可执行文件拷贝至评估板文件系统的任意目录下，执行如下命令查看程序参数说明。

**Target#** ./mqtt_client --help

```
root@IMX8MP-Tronlong:~# ./mqtt_client --help
Usage: mqtt_client [options]

Options:
 -h | --host          The hostname or ip address of the broker to connect to.
 -p | --port          The network port to connect to. Usually 1883.
 -t | --topic         MQTT topic to publish to.
 -m | --message       Message payload to send.
 -M | --mode          MQTT mode [publish|subscribe]
 -q | --quality       quality of service level to use for all messages. Defaults to 0. value: [0, 1, 2]
 -v | --version       Display version information
 --help               Show help content

Example:
  # ./mqtt_client -h 192.168.0.115 -p 1883 -t test -m 'www.tronlong.com'
root@IMX8MP-Tronlong:~#
```

图 4

### 2.2.1 评估板发布/上位机订阅

在上位机执行如下命令，使用 mosquitto_sub 工具订阅 MQTT 主题。

**Host#** mosquitto_sub -h broker.hivemq.com -p 1883 -t test/data

参数解析：

-h：指定 MQTT 服务器；

-p：指定 MQTT 服务器 TCP 端口；

-t：定义 MQTT 主题，可自定义命名。

```
tronlong@tronlong-virtual-machine:~$ mosquitto_sub -h broker.hivemq.com -p 1883 -t test/data
```

图 5

在评估板文件系统执行如下命令发布消息至 MQTT 服务器。

**Target#** ./mqtt_client -h broker.hivemq.com -p 1883 -M publish -t test/data -m 'www.tronlong.com'

```
root@IMX8MP-Tronlong:~# ./mqtt_client -h broker.hivemq.com -p 1883 -M publish -t test/data -m 'www.tronlong.com'
Connect mqtt broker success (broker.hivemq.com:1883).
send message success.
root@IMX8MP-Tronlong:~#
```

图 6 评估板发布

消息发布成功后，上位机将从 MQTT 服务器接收到对应的消息。

```
tronlong@tronlong-virtual-machine:~$ mosquitto_sub -h broker.hivemq.com -p 1883 -t test/data
www.tronlong.com
```

图 7 上位机订阅

### 2.2.2 评估板订阅/上位机发布

在评估板文件系统执行如下命令订阅 MQTT 主题。

**Target#** ./mqtt_client -h broker.hivemq.com -p 1883 -M subscribe -t test/data

```
root@IMX8MP-Tronlong:~# ./mqtt_client -h broker.hivemq.com -p 1883 -M subscribe -t test/data
Connect mqtt broker success (broker.hivemq.com:1883).
```

图 8

在上位机执行如下命令发布消息至 MQTT 服务器。

**Host#** mosquitto_pub -h broker.hivemq.com -p 1883 -t test/data -m www.tronlong.com

```
tronlong@tronlong-virtual-machine:~$ mosquitto_pub -h broker.hivemq.com -p 1883 -t test/data -m www.tronlong.com
tronlong@tronlong-virtual-machine:~$
```

图 9 上位机发布

消息发布成功后，评估板将从 MQTT 服务器接收到对应消息。

```
root@IMX8MP-Tronlong:~# ./mqtt_client -h broker.hivemq.com -p 1883 -M subscribe -t test/data
Connect mqtt broker success (broker.hivemq.com:1883).
recv message: www.tronlong.com
```

图 10 评估板订阅

## 2.3 案例编译

将案例源码目录 src 和 libmosquitto 目录下的压缩包 libmosquitto.tar.gz 拷贝至 Ubuntu 同一目录下，并执行如下命令将 libmosquitto.tar.gz 解压至当前目录。

**Host#** tar -zxf libmosquitto.tar.gz -C ./

```
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_client$ ls
libmosquitto.tar.gz  src
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_client$ tar -zxf libmosquitto.tar.gz -C ./
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_client$ ls
libmosquitto  libmosquitto.tar.gz  src
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_client$
```

图 11

进入工程源码目录，执行如下命令，加载 SDK 交叉编译工具链环境以及编译生成可执行文件

**Host#** source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

**Host#** make

```
tronlong@tronlong-virtual-machine:~/mqtt_client/src$ source /home/tronlong/SDK/
environment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/mqtt_client/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2
 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysro
ot=/home/tronlong/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unus
ed-debug-types  -g -Wall -O2 -I ../libmosquitto/usr/include   -c -o mqtt_client
.o mqtt_client.c
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2
 -D_FORTIFY_SOURCE=2 -Wformat -Wformat-security -Werror=format-security --sysro
ot=/home/tronlong/SDK/sysroots/armv8a-poky-linux  -O2 -pipe -g -feliminate-unus
ed-debug-types  -g -Wall -O2 -I ../libmosquitto/usr/include -o mqtt_client mqtt
_client.o -lmosquitto -lm -L ../libmosquitto/usr/lib
tronlong@tronlong-virtual-machine:~/mqtt_client/src$ ls
Makefile  mqtt_client  mqtt_client.c  mqtt_client.o
tronlong@tronlong-virtual-machine:~/mqtt_client/src$
```

图 12

## 2.4 关键代码

（1） 创建 Mosquitto 实例。

```
mosq = mosquitto_new("client1", true, &params);
if (mosq == NULL) {
    printf("Create new mosquitto client instance failed!\n");
    exit(-1);
}
```

图 13

（2） 设置回调函数。

```
/* Set the connect callback. */
mosquitto_connect_callback_set(mosq, mqtt_connect_callback);
/* Set the disconnect callback. */
mosquitto_disconnect_callback_set(mosq, mqtt_disconnect_callback);
/* Set the publish callback.  This is called when a message initiated with mosquitto_publish has been sent to the broker successfully. */
mosquitto_publish_callback_set(mosq, mqtt_publish_callback);
/* Set the message callback.  This is called when a message is received from the broker. */
mosquitto_message_callback_set(mosq, mqtt_message_callback);
```

图 14

（3） 连接 MQTT 服务器。

```
/* Connect to an MQTT broker. */
ret = mosquitto_connect(mosq, params.host, params.port, keepalive);
if (ret != MOSQ_ERR_SUCCESS) {
    printf("mosquitto connect failed!\n");
    status = -1;
    goto error;
}
```

图 15

（4） 发布消息。

```c
    /* publish */
    if (params.mode == MQTT_PUBLISH) {
        mosquitto_loop_start(mosq);

        while (!g_quit) {
            if (is_connect == false) {
                usleep(100000);
                continue;
            }

            ret = mosquitto_publish(mosq, NULL, params.topic, strlen(params.message), params.message, 0, false);
            if (ret) {
                printf("Error: Publish returned %d, disconnecting.\n", ret);
                mosquitto_disconnect(mosq);
            }

            break;
        }

        mosquitto_loop_stop(mosq, false);
    } else if (params.mode == MQTT_SUBSCRIBE) {
        /* subscribe */
        mosquitto_loop_forever(mosq, -1, 1);
    }
```

图 16

（5） 订阅主题。

```c
void mqtt_connect_callback(struct mosquitto *mosq, void *obj, int rc)
{
    struct _Params *userdata = (struct _Params *)obj;

    if (rc == 0) {
        is_connect = true;
        printf("Connect mqtt broker success (%s:%d).\n", userdata->host, userdata->port);

        /* Subscribe topic */
        if (userdata->mode == MQTT_SUBSCRIBE) {
            mosquitto_subscribe(mosq, NULL, userdata->topic, userdata->quality);
        }
    }
}
```

图 17

# 3 mqtt_sinewave_pub 案例

## 3.1 案例说明

案例功能：使用 libmosquitto(MQTT version 3.1.1 client library)的 API 与 MQTT 代理服务器通信。评估板生成正弦波数据，每秒发送 512 个采样点的数据至 MQTT 服务器；上位机通过 Web 页面从 MQTT 服务器接收到数据后，将会绘制波形。

程序流程图如下图所示。

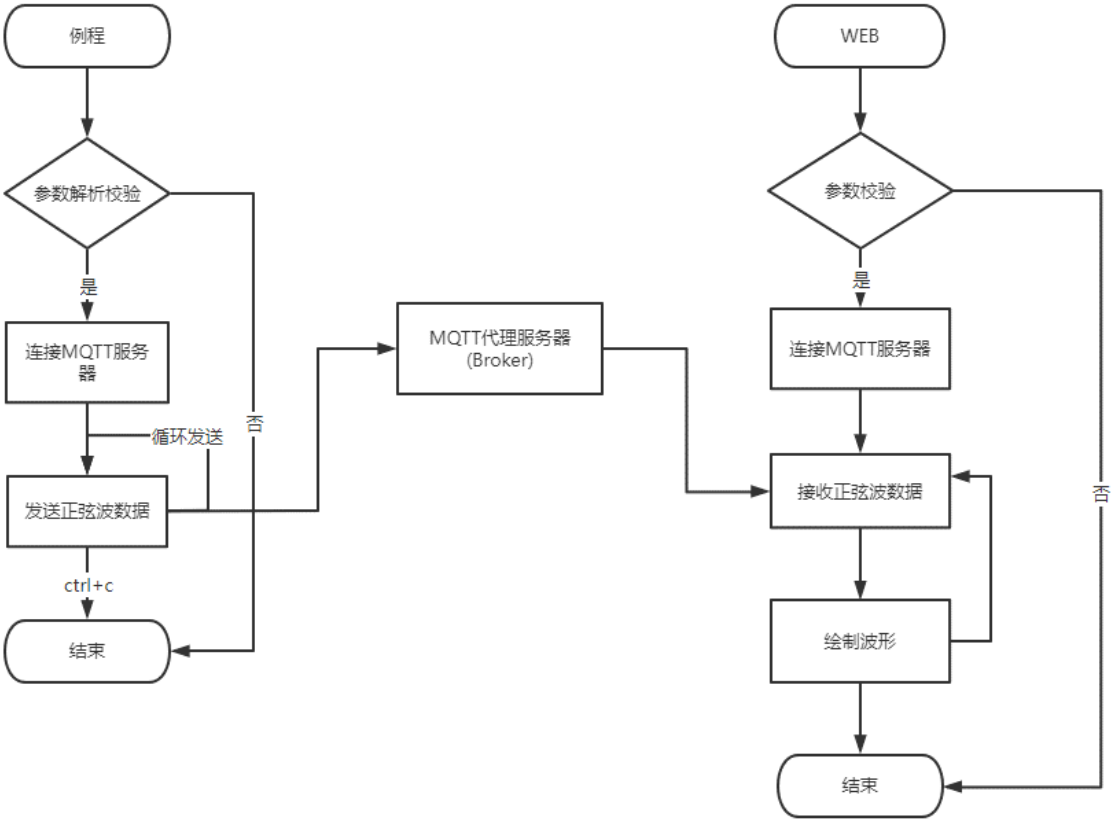


图 18

### 3.2 案例测试

本案例使用公网 MQTT HiveMQ 服务器与上位机 Ubuntu Web 程序通信。请通过网线将评估板千兆网口 ETH1 和上位机连接至公网，确保可正常访问互联网。

评估板启动，将案例 bin 目录下 mqtt_sinewave_pub 可执行文件拷贝至评估板文件系统的任意目录下，执行如下命令查看程序参数说明。

**Target#** ./mqtt_sinewave_pub --help

```
root@IMX8MP-Tronlong:~# ./mqtt_sinewave_pub --help
Usage: mqtt_sinewave_pub [options]

Options:
 -h | --host          The hostname or ip address of the broker to connect to.
 -p | --port          The network port to connect to. Usually 1883.
 -v | --version       Display version information
 --help               Show help content

Example:
  # ./mqtt_sinewave_pub -h 192.168.0.115 -p 1883
root@IMX8MP-Tronlong:~#
```

图 19

执行如下命令运行程序，连接 MQTT 服务器，并发送正弦波数据至 MQTT 服务器。

**Target#** ./mqtt_sinewave_pub -h broker.hivemq.com -p 1883

```
root@IMX8MP-Tronlong:~# ./mqtt_sinewave_pub -h broker.hivemq.com -p 1883
Connect mqtt broker success (broker.hivemq.com:1883).
send message count: 17
```

图 20

评估板程序运行后，将案例"tools\web_mqtt_sub\"目录下 index.html 文件，使用上位机浏览器打开。在弹出的 Web 页面（如下图），依次输入 MQTT 服务器：broker.hivemq.com，端口号：8000，最后点击连接，Web 页面将会从 MQTT 服务器获取正弦波数据并进行波形绘制。

备注：ARM 端 MQTT 通信协议基于 TCP 协议，Web 端 MQTT 通信协议基于 WebSocket 协议，因此使用的端口号不同。

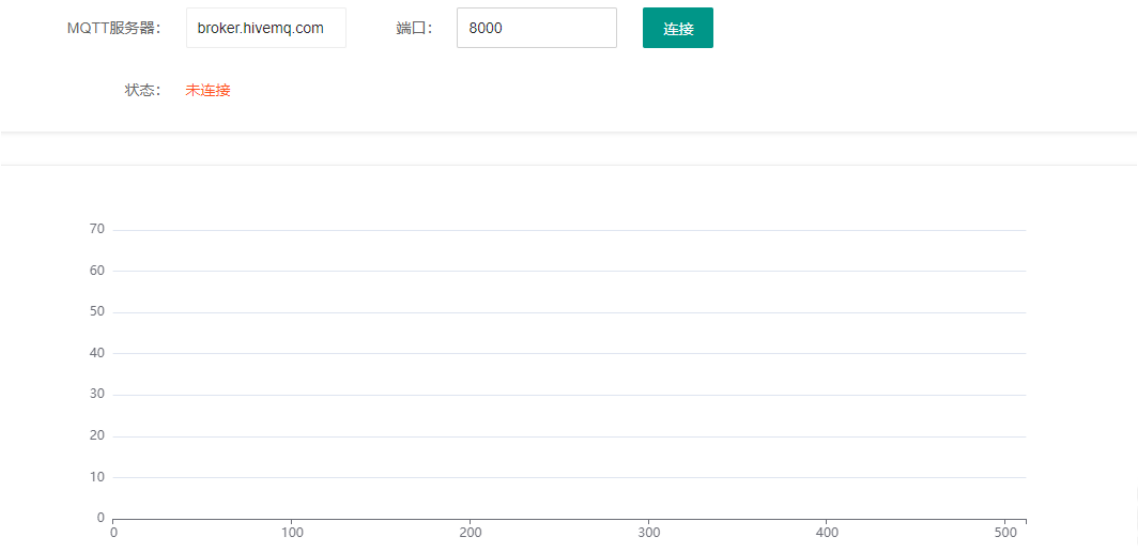


图 21

图 22

### 3.3 案例编译

将案例源码目录 src 和 libmosquitto 目录下的压缩包 libmosquitto.tar.gz 拷贝至 Ubuntu 同一目录下，并执行如下命令将 libmosquitto.tar.gz 解压至当前目录。

**Host#** tar -zxf libmosquitto.tar.gz -C ./

```
tronlong@tronlong-virtual-machine:~/mqtt_sinewave_pub$ ls
libmosquitto.tar.gz  src
tronlong@tronlong-virtual-machine:~/mqtt_sinewave_pub$ tar -zxf libmosquitto.ta
r.gz -C ./
tronlong@tronlong-virtual-machine:~/mqtt_sinewave_pub$ ls
libmosquitto  libmosquitto.tar.gz  src
tronlong@tronlong-virtual-machine:~/mqtt_sinewave_pub$
```

图 23

进入工程源码目录，执行如下命令，加载 SDK 交叉编译工具链环境以及编译生成可执行文件。

**Host#** source /home/tronlong/SDK/environment-setup-armv8a-poky-linux

**Host#** make

```
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_sinewave_pub$ cd src/
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_sinewave_pub/src$ source /home/tronlong/SDK/enviro
nment-setup-armv8a-poky-linux
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_sinewave_pub/src$ make
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -W
format -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-lin
ux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall -O2 -I ../libmosquitto/usr/include   -c -o m
qtt_sinewave_pub.o mqtt_sinewave_pub.c
aarch64-poky-linux-gcc  -march=armv8-a+crc+crypto -fstack-protector-strong  -O2 -D_FORTIFY_SOURCE=2 -W
format -Wformat-security -Werror=format-security --sysroot=/home/tronlong/SDK/sysroots/armv8a-poky-lin
ux  -O2 -pipe -g -feliminate-unused-debug-types  -g -Wall -O2 -I ../libmosquitto/usr/include -o mqtt_s
inewave_pub mqtt_sinewave_pub.o -lmosquitto -lm -L ../libmosquitto/usr/lib
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_sinewave_pub/src$ ls
Makefile  mqtt_sinewave_pub  mqtt_sinewave_pub.c  mqtt_sinewave_pub.o
tronlong@tronlong-virtual-machine:~/mqtt-demos/mqtt_sinewave_pub/src$
```

图 24

### 3.4 关键代码

（1） 创建 Mosquitto 实例。

```
/* Must be called before any other mosquitto functions. */
ret = mosquitto_lib_init();
if (ret != MOSQ_ERR_SUCCESS) {
    printf("mosquitto lib init failed!\n");
    exit(-1);
}

/* Create a new mosquitto client instance. */
mosq = mosquitto_new("client1", true, &params);
if (mosq == NULL) {
    printf("Create new mosquitto client instance failed!\n");
    exit(-1);
}
```

图 25

（2） 设置回调函数。

```
/* Set the connect callback. */
mosquitto_connect_callback_set(mosq, mqtt_connect_callback);

/* Set the disconnect callback. */
mosquitto_disconnect_callback_set(mosq, mqtt_disconnect_callback);
/* Set the publish callback.  This is called when a message initiated with mosquitto_publish has been sent to the broker successfully. */
mosquitto_publish_callback_set(mosq, mqtt_publish_callback);
```

图 26

（3） 连接 MQTT 服务器。

```c
    /* Connect to an MQTT broker. */
    ret = mosquitto_connect(mosq, params.host, params.port, keepalive);
    if (ret != MOSQ_ERR_SUCCESS) {
        printf("mosquitto connect failed!\n");
        status = -1;
        goto error;
    }
```

图 27

（4） 发送数据。

```c
    /* The main network loop for the client. This must be called frequently to keep communications between the client and broker working. */
    while(!mosquitto_loop(mosq, 1, 1) && !g_quit){
        if (is_connect) {
            /* Generate waveform data with different peak, so that web programs can see dynamic waveforms */
            if (count % 2 == 0) {
                generate_sinewave_data(sinewave_data, POINT_NUM, 64);
            } else {
                generate_sinewave_data(sinewave_data, POINT_NUM, 62);
            }

            /* Publish a message on a given topic. */
            mosquitto_publish(mosq, NULL, "data/sinewave", POINT_NUM * sizeof(uint32_t), (uint32_t *)sinewave_data, 0, false);

            count++;

            sleep(1);
        } else {
            sleep(1);
        }
    }
```

图 28

## 更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com